P R U E B A T E C N I C A T O P D E V C R E W

Plan de Arquitectura y Trabajo

Asistente RAG con citas verificables para pólizas de seguros · Faro Seguros

Versión 2.0 (as-built) · 4 de noviembre de 2026

Puesto: Ingeniero AI Full Stack Semi-Senior

Stack: FastAPI · Celery · PostgreSQL + pgvector · React · OpenTelemetry

Estado: Fases 0-6 y post-cierre D30 cerradas · 216 tests · smoke 58/58

Documentación: README.md · DECISIONS.md (30 decisiones) · este plan

Índice

1. Resumen ejecutivo....1

2. Contexto de la prueba y principios rectores....2

3. Objetivos y criterios de éxito....2

4. Diseño de la arquitectura (as-built)....3

4.1 Visión de componentes y despliegue....3

4.2 La estructura hexagonal del monorepo....4

4.3 El flujo de ingesta asíncrona e idempotente....4

4.4 El flujo de consulta agéntica....5

4.5 Modelo de datos e índice vectorial....6

4.6 Los contratos de la API....6

4.7 Observabilidad de extremo a extremo....7

4.8 Seguridad y contenido no confiable....7

5. Decisiones de diseño: del plan al registro real....7

5.1 Las ocho decisiones del plan y dónde aterrizaron....8

5.2 Las treinta decisiones, por bloque....8

6. Ejecución real: fases, cronograma y desviaciones....9

6.1 Las fases tal como se cerraron....9

6.2 El cronograma real....9

6.3 Desviaciones respecto al plan v1.0....10

7. Gestión de riesgos: lo previsto y lo que ocurrió....10

8. Estrategia de entrega, verificación y uso de IA....11

8.1 La checklist de pre-entrega, ejecutada....11

8.2 Trazabilidad requisito-verificación....12

8.3 Registro de uso de IA generativa....12

9. Apéndice: el set de evaluación entregado....13

Nota: este índice se genera mediante campos de Word. Tras editar el documento, haga clic derecho sobre el índice y seleccione «Actualizar campos» para refrescar los números de página.

1. Resumen ejecutivo

Este documento es la versión 2.0 (as-built) del plan que guió la construcción del asistente RAG de Faro Seguros durante la prueba técnica de TopDevCrew. Conserva la estructura de nueve secciones del plan original del 3 de noviembre, pero describe el sistema tal como quedó entregado: un asistente para analistas de siniestros que admite pólizas en PDF o texto plano (hasta 20 MB), las procesa en segundo plano y responde preguntas en lenguaje natural con citas verificables —documento, página y fragmento literal— o con un rechazo explícito cuando la evidencia recuperada no alcanza.

El principio rector se mantuvo intacto de la primera línea del plan a la última corrida de verificación: si la evidencia no alcanza, el sistema se calla. Ese principio se materializa en tres mecanismos que atraviesan todo el diseño: un umbral de similitud de 0.45 aplicado en modo todo-o-nada sobre el mejor resultado recuperado, la verificación carácter a carácter del fragmento citado contra la base de datos, y la degradación en modo cerrado (fail-closed) cuando el proveedor de modelos no responde.

Los números del cierre: siete servicios de Docker Compose que se levantan con un solo comando (make up); 216 tests automatizados que corren sin red ni claves; un smoke end-to-end de 18 secciones y 58 comprobaciones contra el stack real; treinta decisiones de diseño (D01 a D30) documentadas con sus alternativas descartadas; y un set de evaluación con tres pólizas y 21 casos etiquetados que mide precision@k, recall@k y hit@5, además de barrer el umbral de 0.30 a 0.60. Toda consulta termina en uno de tres desenlaces: respondida con citas navegables, rechazada o pidiendo aclaración.

Las desviaciones respecto al plan original fueron numerosas y ninguna fue silenciosa: el proveedor pasó de OpenAI a Cloudflare Workers AI, los embeddings de 1536 a 1024 dimensiones, el umbral de partida de 0.75 a 0.45, y el alcance creció con un bucle agéntico de consulta y un borrado de documentos que el plan había dejado fuera. La sección 6.3 las recoge con su motivo; la lección común es que el plan fijó la disciplina y la ejecución aportó la evidencia.

2. Contexto de la prueba y principios rectores

La prueba se recibió el lunes 2 de noviembre de 2026 a las 23:22 UTC y la ventana de 72 horas expiraba el jueves 5 a la misma hora. La entrega es única: al pegar el enlace se congela el último commit del repositorio y no existe segunda oportunidad. El plan original reservaba tres días de trabajo (unas 20,5 horas efectivas) con puntos de control diarios; la ejecución real comprimió la construcción en dos días intensivos —el día 3 los cimientos y el día 4 el producto completo— dejando el 5 como colchón para el smoke desde cero y la checklist final.

Las reglas del enunciado condicionaron la estrategia tanto como los requisitos técnicos: repositorio público único en formato monorepo, entrevista técnica posterior en la que se recorre el código y se defienden las decisiones, y uso de IA generativa permitido pero exigido con transparencia en DECISIONS.md, incluidos los errores concretos que se corrigieron de lo generado. Las ambigüedades no se preguntaron al evaluador: se resolvieron decidiendo y se documentaron con su alternativa descartada.

La disciplina de alcance del plan se mantuvo como principio rector: el núcleo primero, y el tiempo restante destinado a pulir decisiones, pruebas y documentación. Quedaron fuera de forma consciente la autenticación, el multiusuario, el reindexado, el streaming de respuestas y la recuperación híbrida (D09). La única ampliación del alcance original fue el borrado individual de documentos (D30), que entró en el post-cierre por una necesidad operativa real: limpiar la sonda de telemetría que el propio smoke dejaba en la lista de documentos.

Del informe de optimización de tokens analizado antes de empezar se adoptaron cinco patrones que llegaron intactos a producción: la salida JSON estricta del LLM con system message determinista, los reintentos con timeout y backoff (10, 20 y 40 segundos), la matriz de degradación que alimenta los límites conocidos del README, la idempotencia por huella de contenido y la instrumentación del consumo y la latencia por llamada como atributos del span del LLM. La lección estructural también se mantuvo: ante la caída del proveedor, el sistema se degrada en modo cerrado y nunca genera respuestas sin evidencia validada.

3. Objetivos y criterios de éxito

El éxito se midió con los criterios verificables que el plan fijó por adelantado, alineados uno a uno con el enunciado. La tabla siguiente los confronta con su estado final: los ocho se cumplieron y cada uno tiene una verificación automática que lo demuestra, no una afirmación.

Tabla 1: Criterios de éxito del plan y su estado final verificado.

Criterio (del plan v1.0)	Estado en el sistema entregado	Verificación final
Un comando levanta el stack	make up levanta los siete servicios con healthchecks: db, redis, api, worker, frontend, otel-collector y jaeger	Smoke §1-3: healthchecks y migración desde cero
La subida de un PDF real termina en «listo»	El worker extrae el texto por páginas, lo trocea (800+120) y persiste chunks y embeddings bge-m3 de 1024 dimensiones	Smoke §7-9: ingesta completa contra el stack real
Subir dos veces lo mismo no duplica; reintentar tampoco	Huella SHA-256 única; el contenido viaja por la base, no por el broker; la tarea se despacha tras el commit	Tests de idempotencia + smoke §8
La pregunta con respuesta devuelve citas navegables	La cita trae documento, página y fragmento literal; el clic abre el PDF original en la página exacta (#page=N)	Smoke §12-13 con verificación verbatim

Criterio (del plan v1.0)	Estado en el sistema entregado	Verificación final
Sin evidencia, rechazo explícito sin gastar LLM	Umbral 0.45 en modo todo-o-nada sobre el mejor resultado: por debajo, rechazo sin llamada al modelo	Tests del umbral + smoke §14 (caso fuera de dominio)
Una sola traza HTTP → Celery → LLM	Propagación de contexto W3C por los headers de la tarea; spans de dominio rag.consulta y rag.iteracion	Smoke §16-17: sonda de trazas visible en Jaeger
Suite en CI sin red externa ni claves	216 tests con proveedores simulados; CI: lint + tests (Python 3.12) y build del frontend (Node 24)	GitHub Actions verde en el último commit
README y DECISIONS.md completos	Ejecución desde cero sin pasos ocultos, 30 decisiones con alternativas, límites conocidos y registro de IA	Revisión final + smoke §18

Dos observaciones sobre la tabla. Primera: cada verificación es automática y repetible (make test, make smoke, make eval), de modo que el evaluador no depende de capturas de pantalla. Segunda: los criterios que el plan formuló en términos de «debe» quedaron convertidos en comprobaciones concretas — la traducción de compromiso a verificación es en sí misma un resultado del proyecto.

4. Diseño de la arquitectura (as-built)

4.1 Visión de componentes y despliegue

El sistema desplegado consta de siete servicios de Docker Compose más un proveedor externo de modelos, frente a los seis planificados: el séptimo es el otel-collector, que el enunciado exigía levantar y que la v1.0 había subestimado como opcional. El analista interactúa con el frontend (React 18 + Vite 5 tras nginx, en el puerto 5173), que habla únicamente con la API FastAPI (puerto 8000) a través del proxy /api. La API valida y registra documentos, encola la tarea de ingesta en Redis (6379) y sirve el endpoint de consulta. El worker Celery consume la tarea y ejecuta el pipeline de ingesta contra PostgreSQL 16 con pgvector (5432). Tanto la API como el worker llaman a Cloudflare Workers AI —llama-3.3-70b para redactar, bge-m3 para los embeddings— a través de la misma interfaz de puertos, y ambos exportan trazas OTLP al collector (4317), que las reenvía a Jaeger (16686).

Un detalle de despliegue que la v1.0 no preveía: la máquina de ejecución solo salía a internet por IPv6, y la red bridge interna de Docker es IPv4. La solución —network_mode: host para la API y el worker, documentada en el propio docker-compose.yml— hereda la pila de red del host, y el frontend alcanza la API vía host.docker.internal. Es el tipo de ajuste que solo aparece en una corrida real, y quedó comentado en el archivo para que quien lea el compose entienda el porqué.

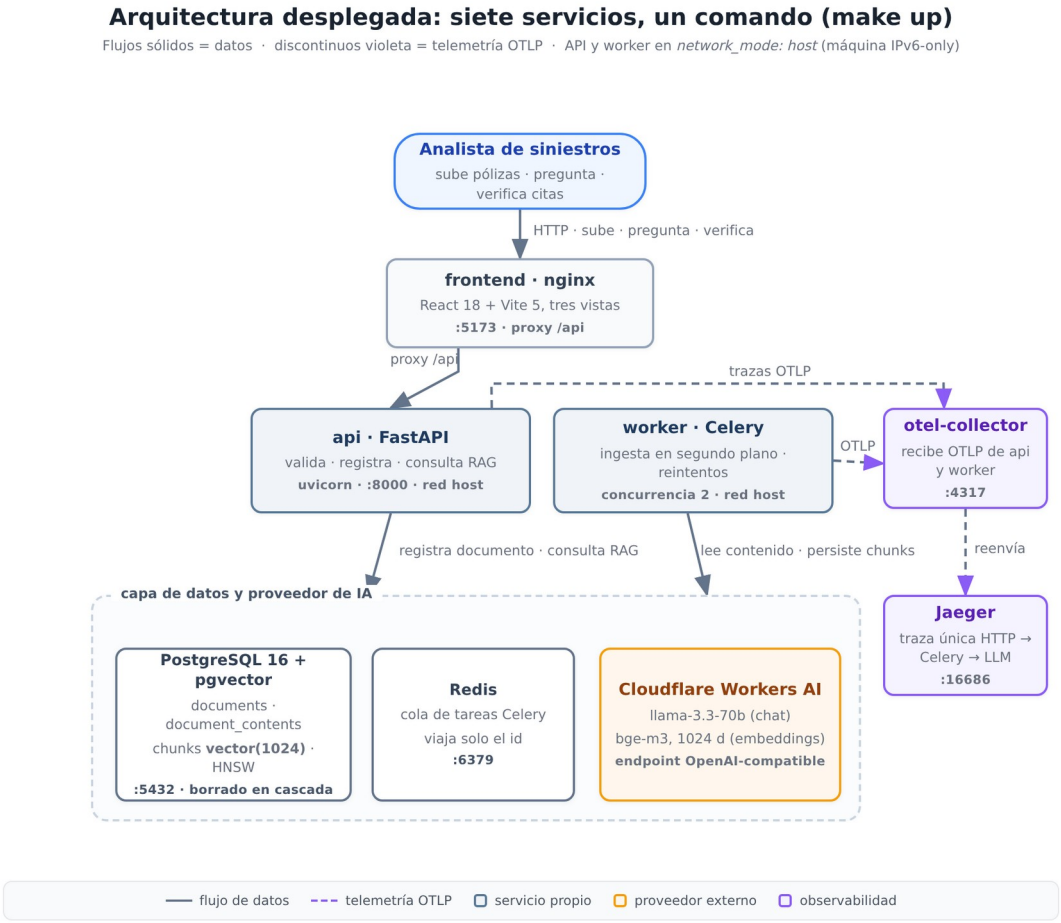


Figura 1: Arquitectura desplegada del sistema entregado: siete servicios, el proveedor externo de modelos y los flujos de datos y telemetría.

4.2 La estructura hexagonal del monorepo

La estructura planificada por capas técnicas (api/, services/, llm/, workers/, db/) evolucionó durante la Fase 0 hacia una arquitectura hexagonal (D01): el negocio en el centro y los frameworks en la periferia. La regla que ordena el código es que las flechas apuntan hacia adentro: dominio y servicios no importan nada de los adapters, y los adapters implementan los contratos (puertos) que el negocio define. La consecuencia práctica son los 216 tests sin red: los dobles (fakes) se inyectan por los mismos puertos por los que entran los adapters reales.

Tabla 2: Estructura del monorepo entregado, por capas de la hexagonal.

Ruta	Contenido
backend/app/domain	Las reglas sin frameworks: chunker por página, decisión de recuperación (umbral), citas literales y la máquina de estados del documento
backend/app/ports	Cinco contratos (llm, embeddings, extraction, retrieval, repository): lo que el negocio necesita de fuera, escrito en sus propios términos
backend/app/services	Los casos de uso: ingesta orquestada (ingestion.py) y consulta agéntica (query.py)

Ruta	Contenido
backend/app/adapters	Las implementaciones: http/ (documentos y consulta), db/ (modelos, repositorio, búsqueda pgvector, sesión), llm/ (Cloudflare, OpenAI y fakes), extraction/ (pypdf, texto plano), tasks/ (Celery) y factory.py (la composición)
backend/app/evaluation	dataset.py (3 pólizas, 21 casos etiquetados), metrics.py (precision@k, recall@k, hit@5) y runner.py (lo que corre make eval, con el barrido de umbral)
backend/app/telemetry	Instrumentación de ambos procesos, propagación HTTP → Celery y apagado limpio con OTEL_ENABLED=false
backend/app/config.py · main.py	Configuración tipada con validación al arrancar (un proveedor mal escrito falla con el listado de valores válidos); /health y el negocio bajo /api
backend/alembic	Dos migraciones versionadas (esquema inicial con vector(1024) e índice HNSW; contenido del documento); se aplican solas al arrancar la API
backend/tests	216 tests por capa más «pins» que congelan lecciones aprendidas; corren sin red ni claves
frontend/src	api.js, App.jsx, DocumentsView, QueryView, Citation.jsx y styles.css: tres vistas, cero grasa
infra · scripts	Configuración del collector; smoke_stack.sh (18 secciones / 58 comprobaciones), las tres pólizas del set y el verificador local de telemetría
raíz del repo	docker-compose.yml (7 servicios), Makefile (up, down, logs, test, lint, smoke, eval), .env.example y el CI de GitHub Actions

4.3 El flujo de ingesta asíncrona e idempotente

La ingesta cubre el requisito más delicado del enunciado: la idempotencia en dos niveles distintos (subida duplicada y reintento de tarea). El flujo planificado llegó a producción como cinco pasos con contratos precisos.

Tabla 3: El flujo de la ingesta, paso a paso.

Paso	Qué ocurre
1 · Subir y validar	El frontend ofrece zona de arrastre y botón; la API rechaza en la puerta lo que no cumple: 415 tipo no permitido, 413 más de 20 MB, 422 sin nombre o archivo vacío
2 · Registrar	Documento y contenido original se guardan juntos en una transacción (documents + document_contents); el estado arranca en pending
3 · Encolar	Solo después del commit, la API encola en Redis el identificador del documento: el contenido ya está a salvo en la base y no viaja por la cola
4 · Procesar	El worker extrae el texto (pypdf o texto plano), trocea por páginas (≈800 caracteres, 120 de solape, palabras enteras), genera los embeddings y persiste todo en una transacción
5 · Cerrar	El documento pasa a ready con sus métricas (páginas, chunks) o a failed con un motivo legible

La máquina de estados vive en el dominio (D04): ninguna capa decide por su cuenta si un cambio de estado es legal.

Tabla 4: La máquina de estados del documento.

Estado	Significado	Transición y nota de borrado
pending	Registrado tras la subida, aún sin procesar	a processing; DELETE responde 409 (la tarea aún no empezó, pero el contrato es único)
processing	El worker ejecuta el pipeline de ingesta	a ready o a failed; DELETE responde 409: la tarea en vuelo moriría contra una fila que ya no existe
ready	Chunks y embeddings almacenados; acepta consultas	final; se puede borrar (204) y la cascada derriba chunks y contenido
failed	Fallo irrecuperable con motivo legible en el estado	final; se puede borrar: es justo la subida rota que se quiere limpiar

La idempotencia de subida se resuelve con la huella SHA-256 del contenido como clave de unicidad: re-subir los mismos bytes devuelve el documento existente (200) sin reprocesar. La idempotencia de reintento, con la persistencia transaccional y una política que distingue fallos transitorios (hasta tres reintentos con esperas de 10, 20 y 40 segundos) de los definitivos (un PDF cifrado falla a la primera, sin reintentar en vano). Desde D30, borrar un documento libera su huella: re-subir los mismos bytes registra un documento nuevo y vuelve a despachar la tarea.

4.4 El flujo de consulta agéntica

El plan contemplaba un RAG de un solo pase (recuperar y generar). El sistema entregado es un bucle agéntico (D19); la diferencia práctica es que el sistema puede detectar que su primer intento no es suficiente y refinarlo, con un techo de tres iteraciones.

Tabla 5: El bucle de consulta del AgenticQueryService.

Paso	Qué hace el agente
1 · Planificar	Decide si la pregunta conviene descomponerla en sub-preguntas, o si es tan ambigua que pide aclaración antes de buscar nada
2 · Recuperar	Embedding de la pregunta (o de cada sub-pregunta) y búsqueda por similitud coseno en pgvector: los cinco fragmentos más parecidos
3 · Decidir	Umbral 0.45 en modo todo-o-nada sobre el mejor resultado: si ni el mejor alcanza, rechazo explícito, sin entregar contexto débil y sin gastar una llamada al LLM
4 · Generar	El LLM redacta apoyado en los fragmentos recuperados y cita un fragmento literal
5 · Verificar	El fragmento citado debe aparecer carácter a carácter en un chunk real de la base; una cita que no cuadra no se emite
6 · Refinar	Si la cita no cuadra o falta evidencia, el agente reformula la búsqueda y reintenta (máximo 3 iteraciones)
7 · Desenlace	respondida (con citas navegables) · rechazada (mensaje fijo) · aclaración (la pregunta era ambigua)

La verificación es la frontera dura del sistema: aunque el modelo se equivocara o alguien intentara manipularlo con el contenido de un documento, no puede emitir texto sin anclaje documental verificado. Los tres desenlaces son visibles en el frontend tal cual, con la cita navegable que abre el PDF original en la página exacta.

4.5 Modelo de datos e índice vectorial

Tres tablas gestionadas con dos migraciones Alembic que se aplican solas al arrancar la API — nunca creación automática de tablas. La dimensión del vector nació del proveedor: bge-m3 produce 1024 dimensiones, y eso fijó la columna vector(1024) desde la primera migración.

Tabla 6: El modelo de datos entregado.

Elemento	Campos clave	Propósito
documents	id (UUID), filename, content_hash (único), mime_type, size_bytes, status, error, page_count, chunk_count, timestamps	Registro del documento, idempotencia de subida y máquina de estados
document_contents	document_id (FK 1:1), bytes del original	El archivo a salvo en la base: el worker lo lee y la cita navegable lo sirve
chunks	id, document_id (FK ON DELETE CASCADE), page_number, chunk_index, text, embedding vector(1024)	Unidad de recuperación; la página viaja desde la extracción porque la cita la necesita
índice HNSW	sobre embedding, operador vector_cosine_ops	Búsqueda por similitud coseno eficiente sin afinar parámetros

4.6 Los contratos de la API

Siete operaciones: las cuatro del plan más el listado (imprescindible para que el frontend muestre estados), el contenido del documento (la cita navegable) y el borrado (D30). Todos los cuerpos de entrada y salida se modelan con Pydantic.

Tabla 7: Los endpoints de la API entregada.

Método y ruta	Entrada	Salida	Códigos
POST /api/documents	multipart: PDF o TXT de hasta 20 MB	id, filename, status	201 creado (con Location) · 200 si el hash existe · 413 / 415 / 422
GET /api/documents	—	lista con id, filename, status, chunk_count	200
GET /api/documents/{id}	identificador en la ruta	estado detallado, error y métricas	200 · 404
GET /api/documents/{id}/content	identificador en la ruta	el archivo original, inline y cacheable (1 h)	200 · 404
DELETE /api/documents/{id}	identificador en la ruta	—	204 · 404 · 409 si está en ingesta

Método y ruta	Entrada	Salida	Códigos
POST /api/query	question; document_id opcional para acotar	status, answer, citations (documento, página, fragmento, score)	200 · 422 · 503 proveedor caído

La cita es un contrato legible por humanos: nombre del documento, página, fragmento literal y puntuación de similitud. El 409 del borrado protege la tarea en vuelo: borrar en plena ingesta dejaría al worker insertando chunks contra una fila que ya no existe, con reintentos ensuciados y trazas ruidosas.

4.7 Observabilidad de extremo a extremo

La pieza de mayor riesgo técnico del plan —la traza única que une la petición HTTP, la tarea Celery y la llamada al LLM— se resolvió con la propagación explícita del contexto W3C: la API inyecta el contexto en los headers de la tarea al encolarla y el worker lo extrae al consumirla. El riesgo se materializó donde no se esperaba: el primer intento instrumentaba el worker en la señal worker_init, cuyo hilo de exportación por lotes no sobrevive al fork de Celery prefork, y los spans se perdían en silencio (D24). La corrección (señal worker_process_init) y la sonda de trazas del smoke —que exige ver su propia traza en Jaeger— convirtieron el requisito en una verificación automática de cada corrida.

Sobre la traza base, los spans de dominio (D27) cuentan la historia semántica: rag.consulta y rag.iteracion registran las puntuaciones frente al umbral, el número de iteraciones y el desenlace. Los logs salen en JSON con el trace_id inyectado para correlacionar cualquier línea con su traza, y todo el subsistema se apaga con OTEL_ENABLED=false: los tests no notan su existencia.

4.8 Seguridad y contenido no confiable

La superficie quedó como el plan la definió, con una adición de la Fase 5. La validación de subidas ocurre en la puerta (tipo verificado por contenido, tamaño máximo de 20 MB, nombre presente); los secretos existen solo como variables de entorno con .env.example completo y auditoría del historial antes de entregar; y la degradación es cerrada: si el proveedor no responde tras los reintentos, el endpoint devuelve 503 con cuerpo estructurado, nunca una respuesta sin evidencia. La adición es el aislamiento del contexto no confiable (D26): los fragmentos viajan delimitados y marcados como datos, nunca como instrucciones, y la validación en el servidor de toda cita impide que una respuesta manipulada cite fragmentos inexistentes.

5. Decisiones de diseño: del plan al registro real

El plan fijó ocho decisiones (D1 a D8) como semilla de DECISIONS.md; el registro final contiene treinta (D01 a D30), porque cada fase añadió las suyas con el mismo formato de contexto, decisión y

alternativas descartadas. Esta sección conecta ambos extremos: primero dónde aterrizó cada decisión del plan, después el mapa completo del registro.

5.1 Las ocho decisiones del plan y dónde aterrizaron

Tabla 8: Trazabilidad de las decisiones D1-D8 del plan v1.0 hacia el registro real.

Plan v1.0	Realidad entregada	Motivo del cambio
D1 · Chunking por página, ~700 + 120	D11 · Por página, 800 + 120, palabras enteras y normalización de espacios	Calibrado con el set de evaluación; las palabras enteras protegen la literalidad de la cita
D2 · Recuperación vectorial pura, top-k = 5, coseno	D02 + D19 · Igual, dentro del bucle agéntico con verificación de citas	El corpus pequeño no pidió híbrida; el bucle añadió refinamiento sin tocar el recuperador
D3 · Umbral en dos capas, partida en 0.75	D13 + D20 · Todo-o-nada sobre el mejor resultado, confirmado en 0.45 end-to-end	Los cosenos legítimos de bge-m3 viven en 0.45-0.60; 0.75 habría rechazado consultas válidas. La segunda capa la cumple la verificación verbatim, no un umbral de generación
D4 · OpenAI: gpt-4o-mini + text-embedding-3-small (1536 d)	D10 · Cloudflare Workers AI: llama-3.3-70b + bge-m3 (1024 d), endpoint OpenAI-compatible	Una sola cuenta y clave para chat y embeddings, capa gratuita suficiente y el único modelo multilingüe del catálogo; las pólizas están en español. OpenAI quedó como preset del mismo adapter
D5 · Idempotencia por hash y reescritura atómica	D03 + D15 + D16 + D17 · Hash único, contenido por la base, despacho tras el commit, reintentos clasificados	La reescritura se volvió transaccional y la política de fallos distingue transitorios de fatales
D6 · Degradación fail-closed	Tal cual: 503 estructurado; nunca se genera sin evidencia validada	Confirmado por diseño y por prueba
D7 · Citas navegables como fragmento expandible	D21 · Fragmento + página + apertura del PDF original en #page=N vía /content (cache de 1 h)	El visor PDF nativo del navegador evita 1 MB de bundle; el contenido es inmutable por el hash
D8 · Recortes conscientes, incluido «sin borrado»	D09 + D30 · El borrado individual sí entró (204/404/409, cascada); el resto sigue como segunda iteración	Lo exigió una necesidad operativa real: limpiar subidas fallidas y la sonda del propio smoke

5.2 Las treinta decisiones, por bloque

El registro completo —con el contexto y las alternativas de cada una— vive en DECISIONS.md; la tabla siguiente es el mapa para orientarse en él.

Tabla 9: El registro D01-D30 agrupado por bloque de proyecto.

Bloque	Decisiones	Esencia
Cimientos (Fase 0)	D01-D09	Hexagonal con puertos propios; pgvector con HNSW coseno; idempotencia por hash; estados en el dominio; Alembic al arrancar; nginx con proxy /api; observabilidad y seguridad desde el día 1; recortes conscientes

Bloque	Decisiones	Esencia
Proveedor e ingesta (Fases 1-2)	D10-D18	Cloudflare Workers AI; chunking por página 800+120; citas literales; umbral todo-o-nada; configuración real por defecto y modo fake explícito; el contenido viaja por la base; reintentos clasificados; despacho tras el commit; el 413 lo cubre la suite
Consulta y frontend (Fases 3-4)	D19-D22	Bucle agéntico con verificación iterativa; umbral confirmado en 0.45; frontend sobrio con citas navegables; el smoke inspecciona cabeceras con GET (curl -D), nunca con HEAD
Observabilidad y evaluación (Fases 5-6)	D23-D29	Telemetría en dos procesos con propagación HTTP → Celery; tres bugs destapados por la primera corrida real; Jaeger always-on; aislamiento del contexto no confiable; spans de dominio; set etiquetado con P@k/R@k y barrido de umbral; cierre con 3 pólizas y 21 casos
Post-cierre	D30	DELETE en cascada, 409 en plena ingesta y smoke que recoge su propia sonda

6. Ejecución real: fases, cronograma y desviaciones

6.1 Las fases tal como se cerraron

El plan organizaba el trabajo en diez bloques (B0 a B9) con criterios de terminación; la ejecución los reagrupó en siete fases más un post-cierre, cada una cerrada con la misma ceremonia: la suite completa y el smoke contra el stack real. Esa doble verificación por fase fue el instrumento de control del proyecto, no el calendario.

Tabla 10: Las fases del proyecto y su verificación de cierre.

Fase	Qué cerró	Verificación
0	Esqueleto: monorepo, compose de 7 servicios, esquema con vector(1024) e índice HNSW, CI	stack arriba + CI verde
1	Dominio puro: chunker por página, umbral, citas literales, máquina de estados	tests unitarios del dominio
2	Ingesta asíncrona idempotente: validación en la puerta, SHA-256, reintentos 10/20/40	smoke con ingesta real
3	Consulta agéntica con verificación de citas; umbral calibrado end-to-end en 0.45	smoke 37/37 (primer cierre completo en verde)
4	Frontend: subida, listado con refresco automático, tres desenlaces, citas navegables	smoke 40/40
5	Observabilidad (propagación, spans de dominio, Jaeger always-on) y evaluación P@k / R@k / hit@5	smoke 48 → 55
6	Cierre: set completo de 3 pólizas y 21 casos, límites conocidos, registro de IA	smoke 55/55
Post-cierre	D30: borrado en cascada (204/404/409), botón con confirmación, smoke auto-limpio	smoke 58/58 · suite 216/216

La progresión del smoke —37, 40, 48, 55, 58— no es anecdótica: cada sección nueva es un requisito convertido en comprobación automática. La fase más larga fue la 5, porque su primera corrida real destapó tres bugs que la suite verde no había visto (D24): la instrumentación del worker en la señal equivocada, la sonda que no despachaba tarea fresca en un stack caliente por la idempotencia, y las aserciones de operación con sufijo donde la librería emite prefijo. La lección quedó institucionalizada como pin de la suite.

6.2 El cronograma real

La Figura 2 sustituye al cronograma de tres días y diez bloques de la v1.0. La forma cambió —los bloques se volvieron fases y los puntos de control se disolvieron en la ceremonia de cierre por fase—, pero la disciplina que el plan fijó se cumplió: los cimientos antes que el producto, la calidad y la documentación con su propio espacio, y margen antes de la entrega.



Figura 2: Cronograma real de ejecución: del plan v1.0 al post-cierre D30, con la verificación de cada fase.

6.3 Desviaciones respecto al plan v1.0

Ninguna desviación fue silenciosa: cada una tiene su decisión en DECISIONS.md. La tabla las resume con su motivo.

Tabla 11: Desviaciones del resultado final respecto al plan v1.0.

Aspecto	Plan v1.0	Realidad entregada	Motivo
Servicios de compose	6	7	El enunciado exigía un colector: el otel-collector entró y Jaeger pasó de opcional a always-on (D25)
Proveedor de modelos	OpenAI: gpt-4o-mini + text-embedding-3-small	Cloudflare Workers AI: llama-3.3-70b + bge-m3	Una cuenta y una clave para todo; capa gratuita; el único multilingüe del catálogo (D10)
Dimensiones del embedding	vector(1536)	vector(1024)	Lo fija bge-m3; la migración nació ya con la dimensión correcta
Umbral de rechazo	partida 0.75, a calibrar	0.45 confirmado end-to-end	Los cosenos legítimos de bge-m3 están en 0.45-0.60; calibración con el set real (D20)
Tamaño de chunk	~700 + 120	800 + 120, palabras enteras	Calibrado con el set de evaluación; las palabras enteras protegen la cita literal (D11)
Consulta	RAG de un pase (recuperar y generar)	Bucle agéntico con verificación iterativa (máx. 3)	El pase único no cubre preguntas multi-paso ni corrige una cita mal atribuida (D19)
Desenlaces	respondida · rechazada	respondida · rechazada · aclaración	El agente detecta la ambigüedad antes de gastar búsqueda
Estructura del backend	capas técnicas (api/, llm/, workers/, db/)	hexagonal (domain, ports, services, adapters)	Los tests sin red lo exigen: los fakes entran por los mismos puertos (D01)
Frontend	React + Vite + TypeScript	React 18 + Vite 5, JavaScript plano	Tres vistas no justifican la deuda de TypeScript (D21)
Borrado de documentos	fuera de alcance (recorte consciente)	DELETE en cascada: 204/404/409	Necesidad operativa real: limpiar la sonda del smoke y las subidas fallidas (D30)
Set de evaluación	12 preguntas (8 con respuesta, 4 sin)	3 pólizas y 21 casos etiquetados (19 + 2)	Cada caso con página y fragmento de evidencia permite P@k, R@k y hit@5 reales (D28/D29)

7. Gestión de riesgos: lo previsto y lo que ocurrió

El plan identificó seis riesgos con su mitigación; los tres primeros condicionaron decisiones de arquitectura, lo que constituyó su primera mitigación. La tabla confronta cada riesgo con lo que realmente ocurrió durante la construcción.

Tabla 12: Los riesgos del plan v1.0 frente a su materialización real.

Riesgo del plan	Qué ocurrió	Cómo se resolvió
La propagación de trazas HTTP → Celery no enlaza (probabilidad alta)	Se materializó, pero no donde se esperaba: el patrón de propagación funcionó; el fallo real fue instrumentar el worker en worker_init, cuyo hilo exportador no sobrevive al fork (D24)	Señal worker_process_init y sonda de trazas del smoke que exige ver su propia traza en Jaeger en cada corrida
Pérdida del número de página en el pipeline (media/alta)	No ocurrió	El chunker nunca cruza páginas (D11) y los tests del dominio lo afirman en cada corrida
Umbral mal calibrado: falsos rechazos o respuestas sin respaldo (media/alta)	El riesgo era real: la partida de 0.75 habría rechazado consultas legítimas	Calibración end-to-end con el proveedor real y confirmación en 0.45 (D20); el falso FAIL de la primera calibración era un bug del parser (jstr no leía booleanos, se añadió jbool)
El LLM alucina citas o fragmentos (media/alta)	No se observó en verificación	Anclaje por fragmento literal verificado carácter a carácter contra la base (D12 + D19): una cita que no cuadra no se emite
PDFs escaneados sin capa de texto (media/baja)	Cubierto por diseño	Estado failed con motivo legible a la primera, sin reintentos en vano (D16)
Flakiness de CI o dependencia de red en los tests (baja/media)	No ocurrió	La suite corre con fakes sin red ni claves; el smoke corre contra el stack real, no en CI

Tres riesgos no previstos sí ocurrieron, los tres destapados por corridas reales y los tres documentados. La máquina de ejecución solo salía a internet por IPv6, y la red bridge de Docker no tenía ruta: la solución fue network_mode: host para la API y el worker, comentada en el compose. Un guard __main__ colocado a mitad de módulo hizo crashear la primera corrida del runner de evaluación con un NameError que la suite no veía —porque importa el módulo en lugar de ejecutarlo—: el guard pasó a ser la última sentencia y un pin de la suite congela la regla para todo módulo ejecutable. Y un CSS del dropzone colapsaba el botón de subida: reproducido 1:1 en el navegador y corregido. La lección transversal: una suite verde no reemplaza una corrida real, y por eso cada fase cerró con las dos.

8. Estrategia de entrega, verificación y uso de IA

8.1 La checklist de pre-entrega, ejecutada

El plan definía una checklist de diez puntos para ejecutar antes de pegar el enlace, con tiempo para corregir lo que fallara. Este es su estado final.

Tabla 13: La checklist de pre-entrega del plan y su estado en la entrega.

Ítem de la checklist	Estado en la entrega
Repositorio público único, monorepo, sin zips ni submódulos privados	Cumplido
CI verde en el último commit (backend Python 3.12: lint + tests; frontend Node 24: npm ci + build)	Cumplido
Smoke desde cero (down -v → up --build → flujo completo)	Cumplido: 18 secciones / 58 comprobaciones
Ningún secreto en el código ni en el historial; .env.example completo	Cumplido
README con ejecución, pruebas, terminado y no terminado, límites conocidos	Cumplido (reescrito con todas las fases)
DECISIONS.md con decisiones, alternativas descartadas y registro de IA	Cumplido: 30 decisiones
Pólizas de ejemplo y preguntas con respuestas esperadas	Cumplido: scripts/data + set etiquetado en app/evaluation
Estados y errores revisados en la interfaz, con mensajes legibles	Cumplido (incluido el botón de borrado con confirmación)
Trazas de ingesta y consulta revisadas en Jaeger	Cumplido (sonda automática en cada smoke)
Entrega con margen antes del cierre de la ventana	Cumplido: el 5 de noviembre quedó como colchón

8.2 Trazabilidad requisito-verificación

La tabla que cierra el ciclo entre lo que el enunciado exige y cómo se demuestra que se cumple; es también el guion natural para recorrer el código en la entrevista.

Tabla 14: Requisitos del enunciado, implementación y verificación.

Requisito del enunciado	Dónde se implementa	Cómo se verifica
API FastAPI con modelos Pydantic	app/adapters/http (documents, query)	Tests de integración de los endpoints + smoke
Ingesta Celery y Redis idempotente	app/services/ingestion.py · app/adapters/tasks	Test de subida doble y de reintento sin duplicados + smoke §8
Esquema Alembic con vector e índice	backend/alembic (dos migraciones)	El smoke verifica la migración desde cero sobre volumen limpio
Endpoint RAG con citas verificables	app/services/query.py (AgenticQueryService)	Smoke §12-13: cita literal verificada contra la base
Umbral de rechazo explícito	app/domain/retrieval.py (0.45, todo-o-nada)	Tests unitarios por encima y por debajo del corte + smoke §14
Frontend con tres funciones	frontend/src (Documents, Query, Citation)	Verificación manual + build en CI + smoke §15
Docker Compose en un solo comando	docker-compose.yml + Makefile	make up; smoke §1-3

Requisito del enunciado	Dónde se implementa	Cómo se verifica
Trazas OTel unificadas y logs correlacionados	app/telemetry + infra/otel-collector-config.yaml	Sonda del smoke §16-17: su propia traza, visible en Jaeger
Seguridad de subidas y secretos	Validación en http/documents · config.py · .env.example	Tests de validación (413/415/422) + auditoría de secretos
CI con lint y pruebas	.github/workflows/ci.yml	Último commit verde antes de la entrega
Documentos y preguntas de evaluación	scripts/data + app/evaluation	make eval: tabla por caso, medias y barrido de umbral

8.3 Registro de uso de IA generativa

El enunciado permite el uso de IA generativa y exige declararlo con honestidad, incluido qué se corrigió de lo generado. La regla de trabajo se mantuvo de principio a fin: la IA acelera el análisis y el esqueleto; toda línea entregada se revisa, se entiende y se puede defender. El registro completo vive en DECISIONS.md; su esencia:

Tabla 15: Registro de uso de IA generativa (herramienta: Z.ai · GLM).

Fecha	Uso asistido	Corrección y aporte propio
2-3 nov	Análisis del enunciado y del informe de tokens; plan de arquitectura y trabajo	Revisión de cada decisión; ajuste del cronograma a la disponibilidad real; contraste de riesgos
3 nov	Esqueleto de cada fase: compose, migraciones Alembic, dominio puro, puertos y adapters	La hexagonal y sus límites de importación se revisaron a mano; los tests de dominio se reescribieron para expresar comportamiento, no estructura
3 nov	Ingesta asíncrona (Celery, idempotencia) y API de documentos	El compose generado dejaba al worker en la red bridge: en una máquina IPv6-only la ingesta moría en reintentos; worker (y después api) pasaron a network_mode: host, documentado en el compose
4 nov	Consulta agéntica y calibración del umbral	El smoke reportaba un FAIL fantasma: el parser jstr no leía booleanos; se añadió jbool (bug del parser, no del umbral)
4 nov	Frontend React + Vite	El CSS colapsaba la caja del dropzone (un label inline tapaba el botón): reproducido 1:1 y corregido con display:block; fechas sin ambigüedad de locale
4 nov	Telemetría OTel	Tres bugs destapados por la primera corrida real (D24): worker_init no sobrevive al fork, el smoke no despachaba tarea fresca en stack caliente y las aserciones usaban sufijo donde la librería emite prefijo
4 nov	Smoke test (secciones 1-18)	curl -I envía HEAD y FastAPI contesta 405: las cabeceras se leen con curl -D del propio GET (D22); una sección llamaba a /content sin el prefijo /api — la suite estaba verde pero la corrida real lo destapó
4 nov	Set de evaluación y pólizas de ejemplo	Los snippets verbatim y las páginas se verificaron contra el texto extraído real antes de commitear (pin PDF ↔ SHA-256 ↔ set); las preguntas se redactaron como las haría un analista, no copiando la cláusula

Fecha	Uso asistido	Corrección y aporte propio
4 nov	Runner de la evaluación	Crash NameError en la primera corrida real: el guard __main__ iba a mitad del módulo y python -m ejecuta main() antes de ligar las definiciones posteriores; guard como última sentencia + pin AST en la suite
4 nov	Borrado de documentos (D30)	El 409 en plena ingesta y el doble check de la cascada (0 chunks en pgvector) son correcciones de diseño: la primera versión habría dejado morir la tarea del worker por FK

Lo que aportó el autor: las treinta decisiones y sus alternativas descartadas, la disciplina de verificación —cada fase cerrada con la suite y el smoke real, no solo con tests—, la calibración con evidencia y la revisión línea a línea de todo lo generado. El patrón del proyecto, que los tests y las corridas reales se corrigen mutuamente, queda escrito en el historial de commits.

9. Apéndice: el set de evaluación entregado

El set cumple una doble función: es el entregable de documentos y preguntas de ejemplo que exige el enunciado y, a la vez, el instrumento de calibración del umbral y del chunking. Se compone de tres pólizas ficticias de Faro Seguros —hogar (poliza-ejemplo.pdf), auto y vida— que comparten vocabulario de oficio a propósito (franquicias, exclusiones, plazos, límites) para que el distractor semántico sea real, como lo es en la vida de un buscador de pólizas.

Tabla 16: Composición del set de evaluación (21 casos).

Póliza	Casos	Ejemplo de pregunta del set
Hogar (poliza-ejemplo.pdf)	10 en dominio	«¿Hasta qué importe máximo cubre la póliza los daños por agua?»
Auto (poliza-auto.pdf)	5 en dominio	«¿Tengo que pagar franquicia si se me rompe una luna del coche?»
Vida (poliza-vida.pdf)	4 en dominio	«¿Qué causas de fallecimiento no están cubiertas por el seguro de vida?»
Fuera de dominio	2	«¿Cuál es la receta tradicional de la tortilla de patatas?» · «¿Cómo se calcula la depreciación de los activos fijos en contabilidad?»

Cada caso en dominio está etiquetado con la póliza, la página y el fragmento literal de evidencia, y esa etiqueta se verificó contra el texto extraído real antes de commitear: la suite congela el pin PDF ↔ huella SHA-256 ↔ set de etiquetas, de modo que nadie —ni la IA— puede etiquetar «a ojo». Sobre esas etiquetas, make eval produce la tabla por caso, las medias de precision@k y recall@k (sobre páginas) y hit@5, además del barrido de umbral de 0.30 a 0.60 con el 0.45 del stack marcado: esa es la herramienta para revisar el corte si el corpus real crece.

Un límite declarado: cada pregunta vive en una sola póliza —la búsqueda se acota a su documento — para que los números sean reproducibles; la recuperación global queda para un corpus real. Las métricas se reportan como evidencia, no como puerta de calidad.